协同验证与性能分析报告

等一个棋迹（GEMwhu） 武汉大学集成电路学院

本阶段在 NEMU 黄金参考模型中完成了 vdot.vv 指令扩展并与香山 RTL 开展 DiffTest 锁步协同验证；同时基于标准 RVV 1.0 指令序列构建等价八项点积基线，系统评测了单链数据相关与多路独立点积两类负载下的周期开销。全部 348 次点积测试与 72 次矩阵乘法测试均通过整核协同仿真：在单链依赖场景下 vdot 相对较快 RVV 基线取得 2.995397 倍周期改善，在 16 项独立点积场景下取得 3.760303 倍吞吐提升。整核波形实测表明，VDotU 内部算术延迟恒为 2 周期，连续独立指令可按 1 周期间隔背靠背发射，而存在写后读依赖的连续 vdot 发射间隔为 7 周期。

赛题要求	本册对应章节与内容
3.1 NEMU 扩展与协同验证	第 7 章参考模型实现、DiffTest 协同机制与多层次验证结果
3.2 已有向量指令组合	第 8.1 节标准 RVV 1.0 等价点积实现及分批归约优化方案
3.3 延迟比较	第 8.2~8.4 节线性拟合评测与加速比结论，第 10 章整核波形时序

7 NEMU 扩展与协同验证

7.1 参考模型实现

在 NEMU 参考模型中，新增的 vdot 译码与执行例程首先校验当前向量 CSR 状态（vtype、vstart 等），确认合法后依次从两个源向量寄存器读取低 8 字节并按有符号 int8 累加乘积；待源操作数全部读出并算完累加和后，再将目的寄存器 128 位整体清零并将结果写入低 32 位。这种“先完整读源、后清零写目的”的顺序建模，天然保证了目的寄存器与源寄存器重名（如 vd=vs1 或 vd=vs2）时不会冲刷尚未读取的源字节。

该扩展受控于编译宏 CUSTOM_VDOT，并在指令修改向量寄存器或 CSR 后主动调用状态同步接口以配合 DiffTest 比对。下方代码截图摘自 NEMU 补丁脚本中的核心算术实现，完整补丁还涵盖了非法配置拦截及 mstatus/vstart 状态维护。

图 7-1 展示了 NEMU 侧的顺序参考实现：待 8 路源字节乘积全部累加进 64 位局部变量 acc 后，再统一清空目标向量寄存器并写入低 32 位，从而规避了源与目的重名时的覆盖冲突。

apply_nemu_patch.py 先读取全部源 再清零并写目的

```
113     int64_t acc = 0;
114     for (word_t idx = 0; idx < VDOT_NLANES; idx++) {
115
116
117       rtlreg_t a, b;
118       get_vreg(id_src2->reg, idx, &a, vtype->vsew, vtype->vlmul, 1, 0);
119       get_vreg(id_src->reg,  idx, &b, vtype->vsew, vtype->vlmul, 1, 0);
120       acc += (int64_t)a * (int64_t)b;
    …
124     for (int i = 0; i < VLEN / 8; i++) {
125       set_vreg(id_dest->reg, i, 0, VDOT_SEW_INDEX, vtype->vlmul, 0);
126     }
127
128     set_vreg(id_dest->reg, 0, (rtlreg_t)(uint32_t)acc,
129              2  , 0  , 0);
130     vp_set_dirty();
131     vstart->val = 0;
```

图 7-1 apply_nemu_patch.py 第 113~120 行与第 124~131 行代码。参考模型采用 C 语言顺序循环建模，与 RTL 侧的 8 路并行乘法加平衡树归约形成独立对照。

补丁脚本路径： nemu/apply_nemu_patch.py；未打补丁的原始 NEMU 源码快照保存在 sources/NEMU/。复现构建时由自动化脚本在独立工作区派生源码副本并打入补丁，原始源码目录自身保持只读洁净状态。

7.2 验证层次与结果

验证层级	核心核验内容	实测验证结果
独立 NEMU 参考模型	遍历 939 组输入，交叉比对三种实现的 128 位完整输出	低 32 位零算术失配，高 96 位零脏数据残留
包装层与算术单元	遍历合法/非法向量配置，开展变异体故障注入测试	包装层 96 项检查全过；41 种人工注入缺陷全部捕获
基础整核功能回归	运行 16 个裸机功能与异常测试程序（软件自检+DiffTest）	16 次整核仿真全部正常收敛通过
性能基准整核回归	点积与 GEMM 性能镜像全程开启结果自检与 DiffTest 比对	348 次点积仿真与 72 次 GEMM 仿真全部通过
整核波形级追踪	逐周期核对分派、源输入、算术输出、分片写回与 ROB 提交	19 条最终有效提交指令的时序与数据完全吻合

在整核协同仿真中，香山 emu 与 NEMU 动态参考库加载完全相同的裸机二进制镜像。每条测试程序一方面将 128 位目的寄存器写回内存并与预置真值做软件自检，另一方面由 DiffTest 框架在每条指令提交时自动比对 RTL 与 NEMU 的通用/向量寄存器及 CSR 状态。由此，软件内存自检、DiffTest 架构状态锁步比对以及 FST 波形追踪三者互为补充，分别从内存落盘值是否正确、指令级架构状态演进是否一致和微架构流水时序是否符合设计三个维度确立验证闭环。

独立运行 NEMU 黄金模型时的比对日志归档于 results/recorded_nemu/（含 patched_vdot.log、patched_vwred.log 与 patched_vwred-g.log）；基础功能集的 16 次整核协同仿真结果位于 results/cluster/20261019/，与后续性能评测的 420 次整核运行分开归档。

下方终端摘要取自 patched_vdot.log，记录了独立 NEMU 参考模型对全部 939 组激励完成 128 位全宽比对的结果（低 32 位零失配、高 96 位零残留）；该日志反映的是参考模型自身的黄金真值核验，整核 RTL 协同结果则由前述 cluster 日志承载。

```
cases=939 mismatches=0 variant=vdot n_inst=1
nonzero_upper_lanes=0 low_mismatches=0 tail_mismatches=0
compare_bits=128
```

7.3 协同检查怎样发现错误

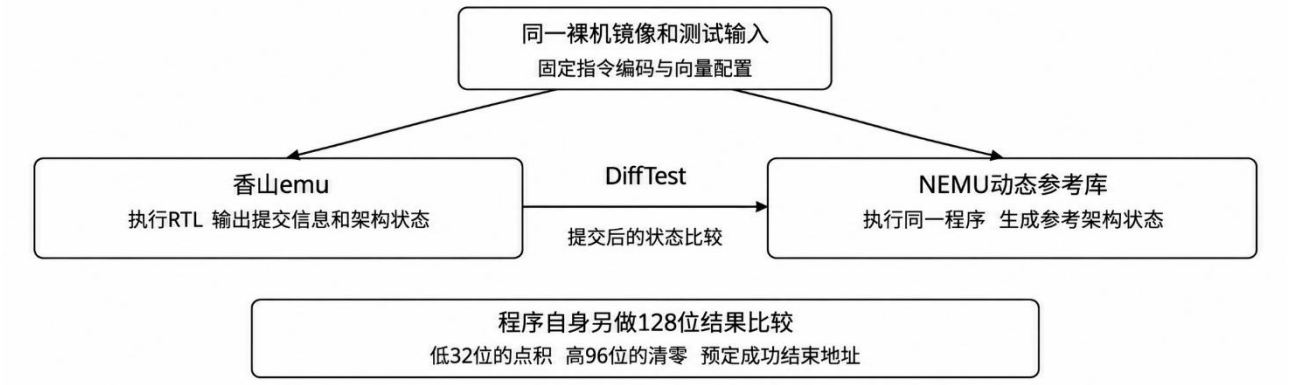


图 7-2 程序软件自检与 RTL-NEMU DiffTest 锁步协同验证架构。同一裸机镜像在香山 emu 与 NEMU 参考库中同步推进，既在提交点比对架构寄存器，又在程序内校验 128 位内存转储值。

启动香山 emu 时通过 --diff 参数挂载编译好的 NEMU 动态参考库（.so）。仿真过程中，每当 ROB 有效提交一条指令，DiffTest 便驱动 NEMU 单步执行并比对双方架构状态；与此同时，裸机程序还会把 128 位目的寄存器转储到内存并与离线期望值逐字核对。这种双重校验机制确保了低 32 位算术精度、高 96 位清零行为以及异常控制流均在整核 RTL 上得到严格验证，避免了仅凭参考模型自测通过就下结论。

由于向量指令的行为高度依赖 vtype 与 vl 等控制寄存器，NEMU 补丁在新增 CUSTOM_VDOT 译码与有符号点积函数的同时，还在所有修改向量配置 CSR 的路径上插入了 csr_difftest_mark_dirty 调用（详见 apply_nemu_patch.py），确保向量控制字变更及时同步给 DiffTest 接口。对于超出第二阶段分册第 4 章约定范围的非法向量配置，参考模型与 RTL 硬件保持一致的异常拦截行为。

在乱序超标量流水线中，因分支预测失败或异常重定向而在后端被冲刷的年轻指令不会写入架构状态。第 10 章在统计波形时，正是依据带翻转位的完整 ROB 表项编号与全局重定向信号，将 52 次分派尝试精确拆分为 33 次中途冲刷与 19 次最终有效提交；DiffTest 锁步比对严格跟随这 19 次有效提交触发。

8 标准 RVV 对照与点积性能

8.1 等价计算与优化方式

为构建公平的标准 RVV 1.0 性能基线，我们使用向量扩宽有符号乘法（vwmul.vv，e8→e16）配合向量扩宽归约求和（vwredsum.vs，e16→e32）来实现数学等价的八项 int8 点积，且要求最终 128 位目的寄存器状态与 vdot.vv 完全一致（高 96 位全零）。在此基础上设计了三个递进版本：原始 RVV 版每次点积均完整执行初值清零与两次 vsetivli 切换；opt1 版将归约零初值寄存器（v28）及目的寄存器高 96 位清零操作外提至循环之前；opt2 版则针对多路独立点积采用四项分批调度，先在 SEW=e8 下连续发射 4 条扩宽乘法，再切换至 SEW=e16 连续执行 4 条扩宽归约，从而大幅摊薄循环内的 vtype 切换开销。

需要注意，标准 RVV 的 vwredsum.vs 仅将累加和写入目的寄存器的第 0 号元素（低 32 位），其余高位元素受尾部策略控制。为了与 vdot.vv 的高 96 位强制清零语义严格对齐，优化基线在进入计时循环前先将各输出向量寄存器整体清零，并在归约时指定尾部保持策略（tu）；倘若只清零提供归约标量初值的源寄存器（v28），目的寄存器高 96 位仍会残留旧值。而在单链依赖场景下，由于后一次点积需要将前一次输出搬移至源寄存器且存在旧目的寄存器依赖，opt1 反而不如原始 RVV 紧凑。

以独立点积 opt1 版本为例，单次八项点积的核心汇编序列如下：先在 e8 配置下完成 vwmul.vv 扩宽乘法，接着用 vsetivli 切换至 e16（tu, mu）执行 vwredsum.vs 累加（v28 存放预先清零的归约初值），最后切回 e8（ta, ma）。完整汇编源码存放在 experiments/throughput/measured_20261103/programs/independent_w16_rvv_opt1_n1024.S。

```
vwmul.vv v16,v31,v30
vsetivli zero,8,e16,m1,tu,mu
vwredsum.vs v0,v16,v28
vsetivli zero,8,e8,m1,ta,ma
```

衡量多路独立点积吞吐时主要以更高效的 opt2 作为标准 RVV 代表。opt2 将 16 项独立点积划分为 4 个批次，每批 4 项：先在 e8 下连发 4 条 vwmul.vv（暂存于 v16、v18、v20、v22），随后执行一次 vsetivli 切至 e16 连发 4 条 vwredsum.vs（结果写入 v0~v3），最后切回 e8。如此一来，每轮 16 项点积循环内的配置切换指令从 opt1 的 32 条骤减至 8 条，核心计算加配置指令总数由 64 条压缩至 40 条。当然，最终端到端周期数仍受限于后端归约单元吞吐率与发射/写回共享资源。

independent_w16_rvv_opt2_n1024.S 第75至85行

```
75   MEASURE_LOOP:
76     vwmul.vv v16,v31,v30
77     vwmul.vv v18,v31,v30
78     vwmul.vv v20,v31,v30
79     vwmul.vv v22,v31,v30
80     vsetivli zero,8,e16,m1,tu,mu
81     vwredsum.vs v0,v16,v28
82     vwredsum.vs v1,v18,v28
83     vwredsum.vs v2,v20,v28
84     vwredsum.vs v3,v22,v28
85     vsetivli zero,8,e8,m1,ta,ma
```

图 8-1 opt2 基线汇编代码中第一批（4 项）独立点积的指令编排。v16~v23 充当扩宽乘积暂存器，v0~v3 保存本批最终点积；后续三个批次复用同一组暂存寄存器。

在编写分批汇编时需统筹规划临时寄存器与结果寄存器的生命周期。只有彼此无数据相关的独立点积序列才能实施这种跨指令分批重排；对于后一项输入直接依赖前一项输出的串行依赖链，无法分批合并，故依赖链对比中选取原始 RVV 与 opt1 两者中的较快成绩作为基线。opt2 完整汇编见 experiments/throughput/measured_20261103/programs/independent_w16_rvv_opt2_n1024.S。

8.2 测量方式

性能测量采用**寄存器热数据+多规模线性拟合**方案：在进入计时窗口前先将测试向量装入寄存器；针对每种实现分别生成循环次数 N=0、64、256、1024 四个版本的裸机程序，每个程序在整核 emu 上独立运行 3 次，取各 N 下的周期中位数后对 N 做最小二乘线性回归。回归直线斜率除以单次循环包含的点积个数，即可得出剔除首尾固定开销、同时保留真实循环控制成本的平均每点积周期数。计时区间首尾通过读取 mcycle 计数器定界，还包括循环结束后的 128 位内存转储与自检指令，这部分固定常数项由斜率法自然剥离，仿真环境的计数器读数步进分辨率为 8 周期。

上述方法重点刻画操作数已驻留向量寄存器时的核心算术与流水调度吞吐，未计入每次从内存冷加载源向量的访存延迟，因此不能直接等同于含复杂访存的完整矩阵乘法或端到端大模型推理性能。全部原始运行日志、生成的汇编程序及重算脚本均保存在 experiments/throughput/measured_20261103/。

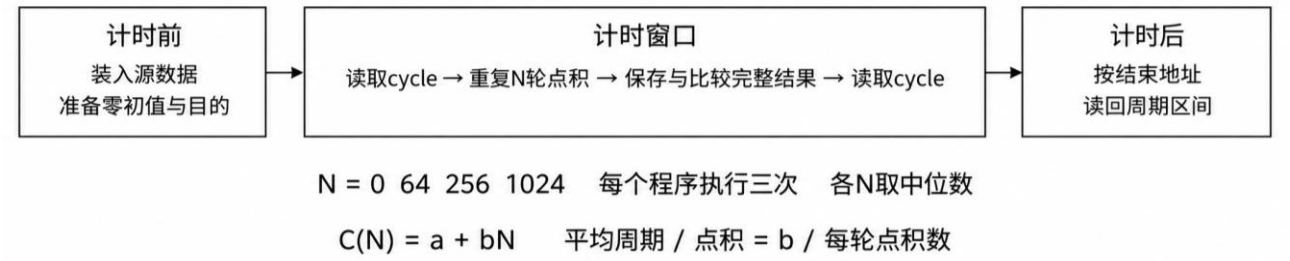


图 8-2 点积性能测试的计时窗口划定与四点线性回归方法示意（严格对应汇编程序中两次读取 cycle 计数器的位置）。

记循环 N 轮的总周期数为 C(N)=a+bN，截距 a 吸收了计数器读取、循环后结果转储比对等与循环轮数无关的固定开销，斜率 b 则代表每增加一轮循环的边际周期成本。对 N=0、64、256、1024 四个中位数点做线性拟合求出 b 后，若每轮循环内包含 W 个点积，则平均每点积耗时 P=b/W，相对加速比定义为 S=P_RVV/P_vdot。本文统一采用包含分支循环控制开销的实测斜率值 P，而不人工扣减空循环周期，以免人为算出低于硬件物理发射间隔的失真数字。

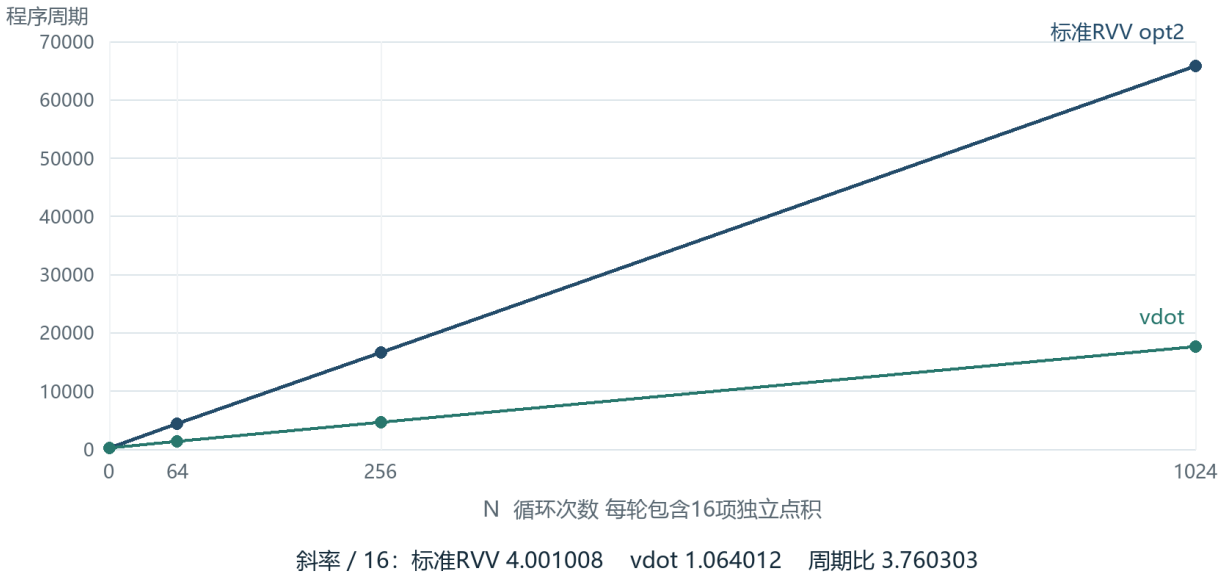


图 8-3 16 项独立点积负载在四个循环规模（N=0、64、256、1024）下的实测周期散点与拟合趋势（源自 348 次点积整核运行数据）。

8.3 延迟与吞吐的直接结论

评测口径与指标	标准 RVV 基线	自定义 vdot	对比结论与说明
单链 RAW 依赖（平均周期/点积）	20.991935	7.008065	端到端延迟改善 2.995397 倍
16 项独立点积（平均周期/点积）	4.001008	1.064012	稳态点积吞吐提升 3.760303 倍
功能单元内部算术延迟（周期）	多指令组合不单列	2	经整核波形中 19 条有效提交指令逐一核实
连续独立指令发射间隔（周期）	本表不单列对比	1	由波形中 4 条连续独立 vdot 指令实测确认

针对赛题 3.3 要求的延迟加速比评估，我们采用单链 RAW 依赖程序进行测定：由于后一条点积指令必须等待前一条点积的结果写回并唤醒后才能发射，测得的平均周期（vdot 为 7.008065 周期，较快 RVV 为 20.991935 周期，改善 2.995397 倍）真实反映了乱序后端的端到端依赖链唤醒延迟与循环控制开销。至于 16 项独立点积测得的 3.760303 倍，反映的是无数据依赖时的稳态流水吞吐率；而波形中观测到的 2 周期则是 VDotU 算术单元自身的纯硬件流水级延迟。这三组指标各自对应不同的微架构观测口径，阅读时应注意区分。

8.4 不同独立点积数量的完整结果

测试负载类型	每轮循环点积数 W	原始 RVV	RVV opt1	RVV opt2	vdot	最快 RVV / vdot
独立点积	1	14.004032	7.028226	7.028226	1.528226	4.598945
独立点积	2	7.497984	4.344758	4.010081	1.179435	3.400000
独立点积	4	5.089718	4.256048	4.003024	1.256048	3.186998
独立点积	8	5.128024	4.252520	4.002016	1.044859	3.830198
独立点积	16	5.022429	4.251260	4.001008	1.064012	3.760303
单链 RAW 依赖	1	20.991935	27.991935	不适用	7.008065	2.995397

上表各列耗时单位均为处理器周期/八项点积，最右侧加速比统一取相同测试负载下标准 RVV 各版本中的最快成绩除以 vdot 耗时。其中，16 项独立点积的吞吐比为 4.001008 / 1.064012 = 3.760303，单链依赖的延迟比为 20.991935 / 7.008065 = 2.995397。表中保留六位小数是为了忠实呈现线性回归的计算结果，硬件底层 mcycle 计数器本身的采样粒度仍为 8 周期。

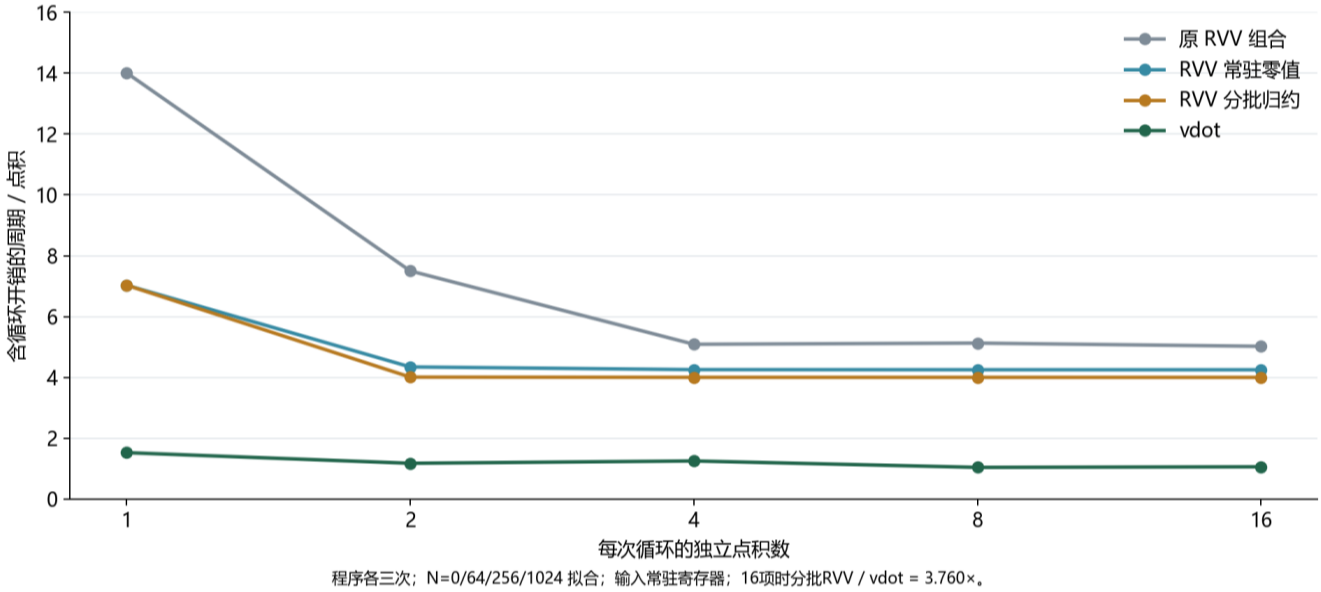


图 8-4 不同每轮独立点积数量（W=1、2、4、8、16）下的平均周期对比曲线（汇总自 348 次整核 DiffTest 仿真，单链依赖数据见表 8-4 最后一行）。

数据位置： experiments/throughput/measured_20261103/checked_local/comparison.csv 与 optimized_speedup.csv。注：文中提到的 348 次执行是指不同指令实现方案、不同展开规模与 4 个 N 值各重复运行 3 次的仿真总场次（116 个镜像×3 次）。

9 GEMM 应用比较

9.1 测试范围

为进一步考察自定义点积指令在实际算子中的表现，我们选取 3×5×13（非对齐奇异尺寸）、16×16×16 和 32×32×32 三种矩阵规模，对比了四种通用矩阵乘法实现：标准 RVV 点积式、标准 RVV 外积式、vdot 直接替换版以及 vdot 四列展开版。每种实现分别按仅核心矩阵计算与包含右矩阵 B 转置/重排准备两种口径计时，共生成 24 个裸机程序、各跑 3 次，累计完成 72 次整核协同仿真。

在数据布局上，RVV 点积式与两种 vdot 实现均需沿内积维度 K 连续加载数据，因此要预先将右矩阵 B 按列重排（转置）为连续内存；而标准 RVV 外积式（基于 vwmul.vx 与 vwmacc 思想）沿输出行方向 N 展开向量化，天然支持按行连续读取原始矩阵 B。在含 B 准备计时口径下，测试框架对 RVV 外积式也一视同仁地计入了一次矩阵 B 的连续内存拷贝开销；若在实际推理引擎中原地直接读取原始矩阵 B，外积式省去这次拷贝后的领先优势还会进一步扩大，本次未单独剥离该项测量。

9.2 全部比较结果

矩阵规模 M×N×K	计时口径	RVV 点积式	RVV 外积式	vdot 直接版	vdot 四列展开
3×5×13	仅核心计算	1496	736	760	984
3×5×13	含矩阵 B 准备	1896	880	1208	1416
16×16×16	仅核心计算	10288	3088	3832	3704
16×16×16	含矩阵 B 准备	10824	3408	4480	4296
32×32×32	仅核心计算	76832	21480	30448	21624
32×32×32	含矩阵 B 准备	78592	22512	31984	23032

表中数据为 3 次整核运行的中位数（计时器步进精度为 8 周期）。可以看到，在全部 6 组测试条件下，标准 RVV 外积式的总周期数均低于两种 vdot 点积式实现：以 32×32×32 方阵为例，vdot 四列展开版在仅计算口径下比 RVV 外积式多耗费 144 周期，在含 B 准备口径下多耗费 520 周期。这一实测结果说明：虽然 vdot 显著快于同样采用内积算法的 RVV 点积式实现，但面对充分利用向量通道并行度且免去矩阵转置的标准 RVV 外积式算法时，单纯替换内积指令并不能在完整 GEMM 算子上取得领先。

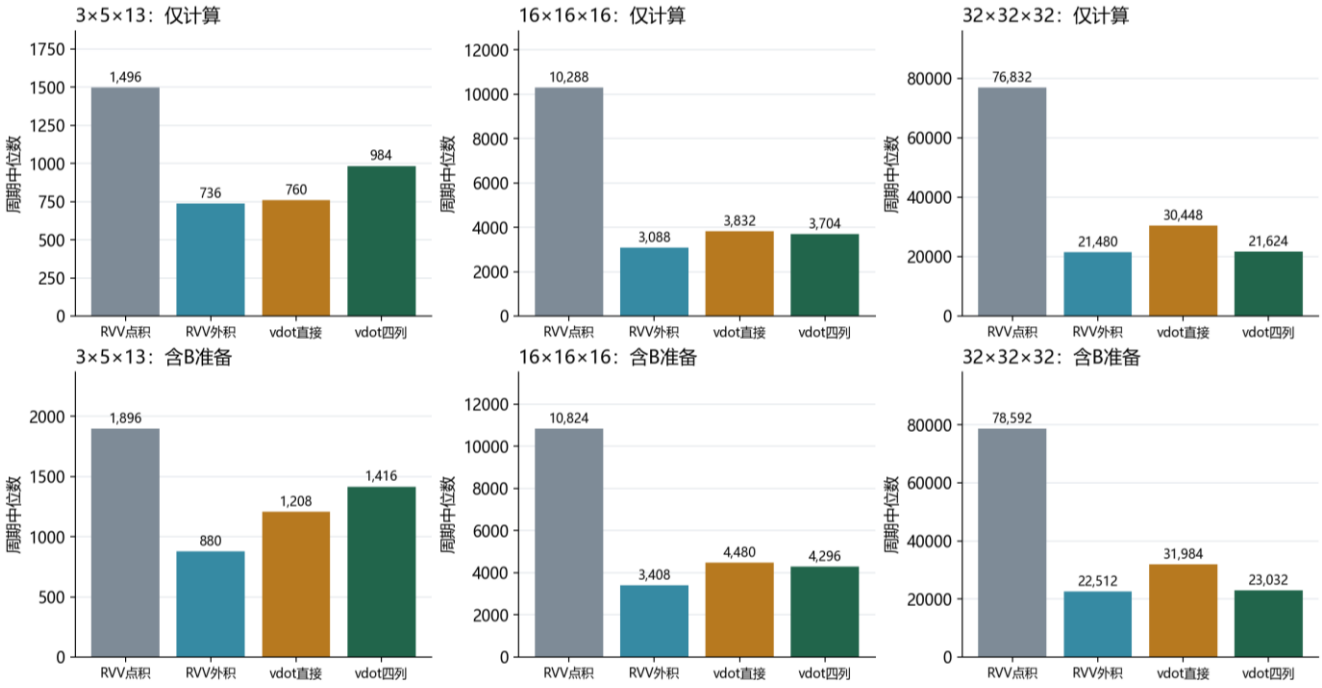


图 9-1　三种矩阵形状下四种 GEMM 实现的整核周期对比（上方为仅计算口径，下方为含右矩阵 B 准备口径）。

拿 32×32×32 方阵来说，我们最优的 vdot 四列展开版相比标准 RVV 外积式在纯计算阶段慢了约 0.67%（+144 周期），计入矩阵 B 重排后慢了约 2.31%（+520 周期）。这清晰地揭示了一个微架构优化规律：局部算术指令条数的缩减并不会无条件转化为完整算子的端到端加速；要想让点积加速单元在矩阵算子中真正胜出，必须协同解决操作数多列复用、跨 K 分块原位累加以及访存重排开销等系统级瓶颈。

具体剖析原因：当前 vdot.vv 每次仅产出单个 32 位部分和（标量结果置于 vd 低 32 位），当内积维度 K>8 时需依赖额外的标量/向量指令跨分块累加，且右矩阵 B 必须预先按列重排；反观 RVV 外积式，单条向量乘加指令即可同时更新一行内的多个输出列元素，向量寄存器利用率与访存连续性均占优。端到端周期数是由算法映射、内存布局与微架构流水共同决定的综合结果。

归档路径：applications/gemm/measured_20261103/checked_local/comparison.csv 及同目录下的汇编源码与原始日志。受限于仿真时长，本次实验仅覆盖了上述中小尺寸矩阵与热缓存状态，尚不足以直接外推至超大规模矩阵或完整 LLM 推理负载。

10 完整处理器波形分析

10.1 首条 vdot 指令

整核录波程序在单个镜像中串联了 11 条代表性算术测试指令、4 条 RAW 依赖链指令以及 4 条独立发射指令。以首条执行的 CASE_01 为例，其指令编码为 0xb284a257（PC=0x80000044），两个源向量寄存器的低 8 字节均填入 0x01，理论点积和为 8，目的寄存器高 96 位应全为 0。

流水线事件	处理器周期 C	波形观测详情
指令分派（Dispatch）	4243	指令 PC（0x80000044）机器码与 ROB 编号完全匹配
算术单元输入（Issue）	4312	两个源向量端口的低 8 字节均送入 0x01
算术单元输出（Result）	4314	128 位输出端产生 0x08，高 96 位及高字节全为 0
物理寄存器写回（Writeback）	4315	四个 32 位写回分片同时拉高写使能
架构提交（Commit）	4316	指令在 ROB 中完成按序退休提交

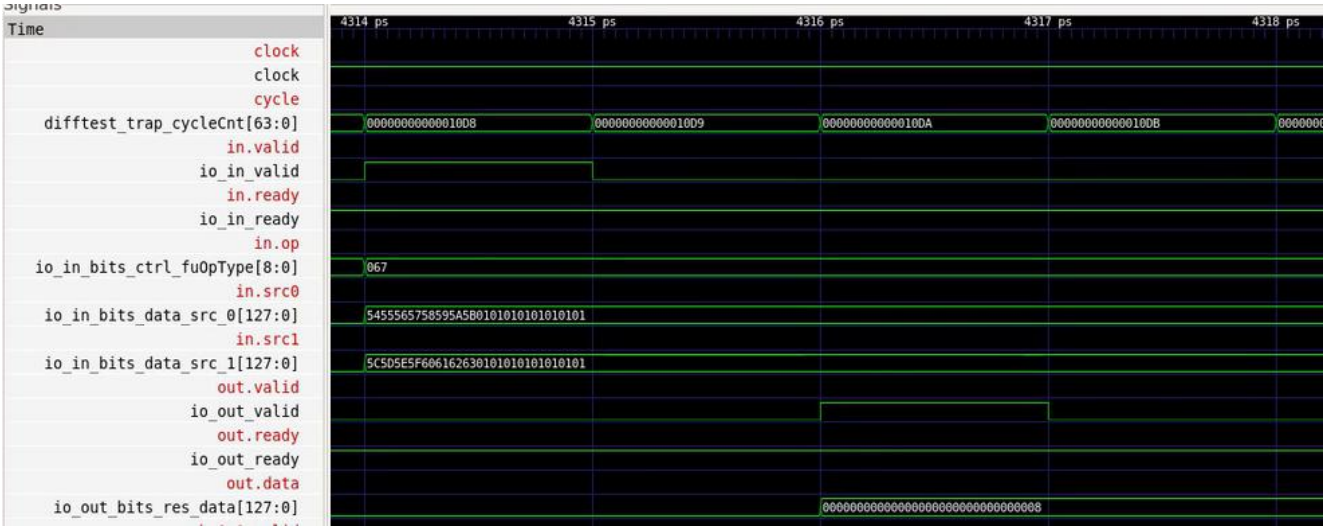


图 10-1　整核仿真中首条 vdot.vv 指令的实测波形局部

对应的原始 FST 波形文件存放在 results/vdot_full_core_20261102/vdot.fst，由脚本提取的逐指令流水节拍明细表位于同目录 checked_wave/vdot_executions.csv。图 10-1 中的端口数值与表 10-1 所列的周期节点及算术输出完全吻合。

10.2 依赖链与独立执行

纵观波形中全部 19 条最终提交的 vdot.vv 指令，其在 VDotU 内部的算术执行耗时无一例外均为 2 个周期，从操作数送入功能单元到 ROB 完成架构提交的总跨度则为 4 或 6 个周期。其中，4 条串行 RAW 依赖链指令进入运算单元的时刻依次为周期 5087、5094、5101、5108，呈现极规整的 7 周期间隔；而 4 条无相关独立指令进入运算单元的时刻依次为周期 5210、5211、5212、5213，实现了间隔 1 周期的背靠背连续吞吐。

测试序列与指令序号	单元输入周期	单元结果周期	写回控制周期	ROB 提交周期
RAW 依赖链 1	5087	5089	5090	5091
RAW 依赖链 2	5094	5096	5097	5098
RAW 依赖链 3	5101	5103	5104	5107
RAW 依赖链 4	5108	5110	5111	5112
独立发射序列 1	5210	5212	5213	5214
独立发射序列 2	5211	5213	5214	5215
独立发射序列 3	5212	5214	5215	5216
独立发射序列 4	5213	5215	5216	5217

观察表 10-2 可以发现，依赖链第 3 条指令（输入周期 5101，结果周期 5103）直到周期 5107 才完成架构提交，比其余指令多等了 2 个周期。这源于 ROB 按序提交队列等待前序指令退休，并不影响 VDotU 内部固定的 2 周期算术流水，也未拖慢第 4 条指令在周期 5108 准时获得转发唤醒并进入运算单元。此外，独立序列展现的 1 周期背靠背发射间隔属于短突发（4 条）下的理想流水观测值；若在长循环中持续运行，平均每点积耗时还需计入循环分支与指令获取开销（即第 8 章实测的 1.064012 周期/点积）。

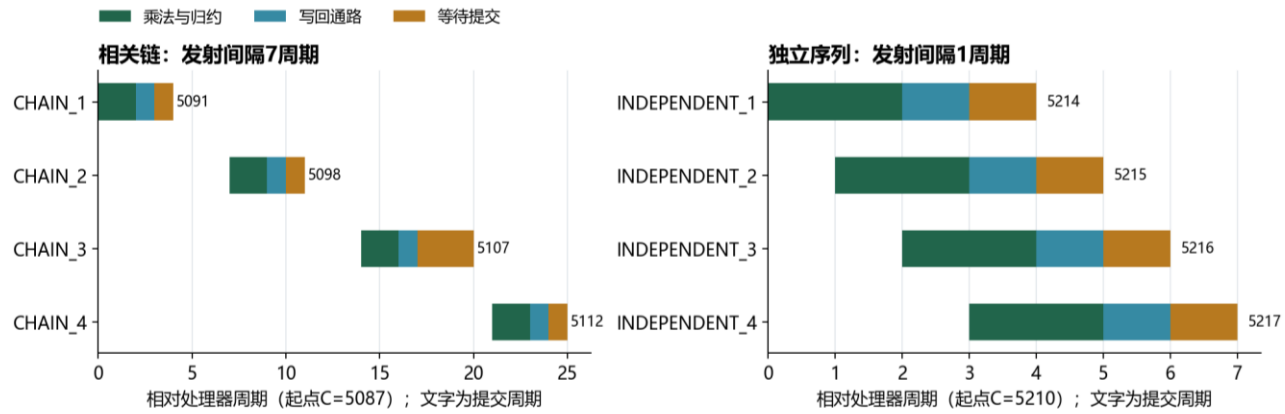


图 10-2　基于实测 FST 波形节拍数据绘制的 4 条 RAW 依赖链（左）与 4 条独立指令序列（右）流水甘特图，柱状条右侧数字标明了 ROB 最终提交周期。

在整核乱序执行过程中，波形追踪脚本共捕获到 52 次匹配目标 PC 的指令分派事件，其中 33 次因前序异常或分支重定向被中途冲刷，最终真正到达提交阶段的为 19 条。为确保统计精确，分析脚本联合比对了带翻转位的完整 ROB 编号与全局重定向有效信号，从而避免了仅靠 PC 地址或裸 ROB 索引号去重带来的误判。详细分派与冲刷记录见 checked_wave/dispatch_attempts.csv 与 issue_intervals.csv。

11 面积、时序与扩展

11.1 同条件模块估算

2026 年 10 月 30 日，我们使用开源逻辑综合工具 Yosys 配合静态时序分析工具 OpenSTA，基于 Nangate 45nm Open Cell Library（typical 工艺角），在完全一致的综合脚本约束下对算术核、完整 VDotU 以及香山原有的两个向量单元进行了横向逻辑综合评估。下表所列面积均为标准单元映射后的组合与时序逻辑面积之和，尚未计入后端布局布线的布线通道与缓冲器开销。

综合评估模块	标准单元数（Cells）	逻辑综合面积（μm²）	最大路径到达时间（ns）
点积算术核（平衡二叉树）	5135	6426.028	1.1951
点积算术核（Wallace 压缩树）	5310	6612.494	1.1275
当前完整 VDotU（含包装层）	7700	14614.306	6.4400
香山原生 VIMacU	53412	68759.936	15.4123
香山原生 VIAluFix	33388	41856.962	7.9678

在同一综合基准下，完整 VDotU 模块的标准单元面积（14614.306 μm²）仅相当于香山原生向量乘加单元 VIMacU 的 21.25%，或向量定点算术单元 VIAluFix 的 34.91%。需要指出，原生模块承载了多种位宽与复杂 RVV 指令模式，而 VDotU 专注于固定八项 int8 点积，此处对比旨在提供直观的模块级资源量级参照；由于本次未对昆明湖 V2 全芯片做整核逻辑综合，故不折算占全核总面积的百分比。

时序分析统一挂载 10 ns（100 MHz）理想时钟约束，未纳入物理连线寄生 RC、高扇出网络缓冲树（CTS/Buffering）及门级尺寸微调（Gate Sizing）。从 OpenSTA 报告来看，完整 VDotU 的关键路径（6.4400 ns）实际落在外围包装层的控制与异常译码逻辑上，而非内部点积算术核（仅约 1.13~1.20 ns）；因此表中的逻辑级路径延迟仅供模块微架构横向对比，不可直接取倒数当作流片后的物理最高主频。综合脚本、网表报告及约束说明归档于 synth/results_20261030_nangate45/summary.csv 与 synth/README.md。

11.2 归约结构与流水选择



图 11-1 Nangate 45nm 同条件逻辑综合下平衡加法树与 Wallace 压缩加法树算术核的面积及关键路径对比（注：路径时间为逻辑综合报告的组合路径到达时间，非整核物理时钟周期）。

对比两种归约树实现：采用 Wallace 压缩树（tree=1）的算术核面积为 6612.494 μm²，较平衡二叉树（tree=0，6426.028 μm²）微增 2.90%，而算术核内部关键路径延迟从 1.1951 ns 缩短至 1.1275 ns（改善 5.66%）。不过，由于完整 VDotU 模块当前的时序瓶颈位于外层控制包装逻辑（6.4400 ns），算术核内部削减的 0.0676 ns 并不会缩短整个模块的综合关键路径，也无法减少既定的 2 周期流水延迟。

综合考量后，当前接入整核的 VDotU 默认选用了面积更省、连线更规整的平衡二叉树方案（tree=0），并以此完成了全部整核功能与性能验证。压缩树方案已作为参数化可选项保留在 Chisel 生成器中；未来若针对高频物理实现优化外围控制路径后，可随时切换至压缩树并重新评估整核时序。

11.3 后续设计方向与扩展成本

结合第 9 章 GEMM 实测暴露出的瓶颈，后续微架构演进有两条清晰路径：一是在现有八项点积基础上引入原位乘累加语义（读取旧 vd 低 32 位并与八项点积相加），从而省去跨 K 分块时的额外累加指令；二是向更高并行度或 RISC-V Zvqdotq 标准草案靠拢，支持在单个向量寄存器内按每 4 字节（32 位）一组并行完成 4 组四项 int8 点积乘累加，大幅提升右矩阵多列数据的寄存器复用率。当然，这两类改动均需重新打开旧 vd 读取通路，并相应增加重命名依赖跟踪与数据对齐开销。

至于将算术流水改为单周期或三级流水、与现有 VIMacU 复用底层 8×8 乘法器阵列、或者做双编码兼容分流等设想，目前仍处于微架构方案探讨阶段，尚未开展对应的整核 RTL 改版与协同仿真。后续若推进这些改动，评测时同样应将标准 RVV 外积式基线、矩阵重排预处理及写回开销一并纳入端到端考核，避免单纯比拼局部算术指令条数。

微架构扩展方向	数据通路与写回格式变化	需重新评估的硬件与验证代价
VLEN=256，仍算 8 路 int8	维持 19 位累加；低 32 位写结果，高 224 位清零	256 位源读取总线、写回分片扩充及 VRF 端口适配
VLEN=256，扩展为 16 路 int8	有效乘加通道翻倍，无损累加位宽升至至少 20 位	16 路并行乘法器、归约树深度增加、级间寄存器与时序收敛
支持 8 路 int16 有符号点积	正极值达 8589934592，无损累加至少需 35 位精度	输入字节重切片、16×16 乘法器阵列，且原 32 位写回域无法容纳需扩宽
跨 K 分块原位乘累加（MAC）	引入旧 vd 低 32 位作为第三源操作数，需界定溢出语义	恢复第三源操作数读端口、RAW 依赖唤醒链及 NEMU 参考模型同步修改

此外，当前 Chisel 生成器的输入切片逻辑仍按 8 位字节硬编码实现。虽然配置类中预留了 SEW 字段，但若要真正扩展支持 8 路 int16 点积（单次点积正上限达 8589934592，无损累加至少需 35 位有符号精度），不仅需要重构输入多路选择与 16×16 乘法阵列，还必须重新定义超出 32 位字长的目的寄存器写回格式。以上均属面向未来的架构演进分析，本次交付的整核实装版本严格锁定为固定八项 int8 点积。

12 复现与资料导航

12.1 工作副本与环境

本章提供基于交付包归档材料的离线复查与整核重跑指南。为保护原始证明数据不被意外覆盖，所有复现、重编译及重算操作均应在新建的独立工作目录中进行：请先准备磁盘空间充足的 Linux 环境，将交付包中的相关证明材料完整拷贝至个人工作区，随后在该可写副本内执行脚本。

编译与仿真所需的工具链版本、香山与 NEMU 源码 commit 信息已列于第一阶段分册第 2 章及 source_revisions.csv。特别提醒，编译完整香山 Verilator 仿真器对主机内存要求极高，复现时请根据所在服务器的实际硬件规格合理调整并发度与堆大小。

12.2 已有结果复查

进入工作副本的相关证明材料目录后，可直接运行以下两条轻量级 Python 脚本对已归档的实验日志进行离线复核與统计重算（请确保 --out 指向全新的输出路径，无需重新启动耗时的整核 Verilator 仿真）：

```
python3 -B scripts/check_recorded_results.py
python3 -B scripts/check_measurements.py --out ../current-measured-new
```

其中第一条命令遍历核验 16 个基础功能测试程序的整核运行日志，第二条命令则读取 420 份性能测试原始输出并重新计算线性回归斜率与中位数。关于整核波形的逐指令提取数据，可直接查阅 results/vdot_full_core_20261102/checked_wave/；第 12.5 节进一步给出了从头生成并重跑全部性能基准程序的完整命令序列。

12.3 完整处理器构建

若需在已配好工具链的计算节点上从头编译香山整核仿真器并重跑基础测试集，可在工作副本内依次执行以下命令。其中 RUN_ROOT 必须指向工作副本以外的新建空目录；执行前请结合机器内存容量调整 BUILD_JOBS 与 EMU_COMPILE_JOBS 并发参数：

```
source scripts/env.sh
export RUN_ROOT="$HOME/vdot-work/full-new"
export BUILD_JOBS=2
export EMU_COMPILE_JOBS=2
python3 -B run_latency.py check
python3 -B run_latency.py build --work "$RUN_ROOT" --jobs 2 --java-heap
64G
python3 -B scripts/run_full.py --run-root "$RUN_ROOT" --workers 4 --
repeat 3
```

12.4 主要文件

资料类别	归档路径
源码版本快照清单	source_revisions.csv
第一阶段三层问候语输出	stage1/measured_20261102/
原生 vadd 实测波形	waveforms/vadd.fst
vdot Chisel 源码与后端补丁	rtl/chisel/vdot/ 与 rtl/xiangshan-patch/
基础功能测试集与集群结果	tests/、programs/、results/cluster/20261019/
NEMU 黄金模型扩展补丁	nemu/apply_nemu_patch.py
点积与 GEMM 性能评测数据	experiments/throughput/measured_20261103/； applications/gemm/measured_20261103/
vdot 整核波形与节拍明细	results/vdot_full_core_20261102/
Nangate45 逻辑综合与时序报告	synth/results_20261030_nangate45/
本地部署与计算节点执行日志	执行日志/

12.5 当前性能程序的生成与执行

以下性能重跑命令同样需在可写工作副本中执行，且 --out 指定的目录必须为尚未创建的新路径。操作前须先按第 12.3 节完成整核构建，然后从构建输出目录的 build_manifest 文件中查出当前机器实际生成的 emu 可执行文件路径与 NEMU 动态库（.so）路径，分别赋给环境变量 EMU 和 REF（切勿直接照搬其他机器遗留的绝对路径）：

```
python3 -B experiments/throughput/study.py generate \
  --optimized --out ../dot-programs-new
python3 -B experiments/throughput/study.py execute \
  --programs ../dot-programs-new --emu "$EMU" --library "$REF" \
  --out ../dot-results-new --repeat 3 --workers 4 \
  --cycle-limit 250000 --seconds-limit 7200
python3 -B scripts/check_measurements_opt.py \
  --results ../dot-results-new --out ../dot-analysis-new
```

点积测试生成器默认遍历 N=0、64、256、1024 四种循环规模；传入 --optimized 参数后会一并生成原始 RVV、opt1、opt2、vdot 以及控制流对照程序（共 116 个裸机程序，每个重复执行 3 次，合计 348 次仿真）。在正式启动大批量仿真前，可先运行 study.py estimate 预估总耗时（传入与 execute 相同的并发数与超时阈值）。

```
python3 -B applications/gemm/rvv_benchmark.py generate \
  --out ../gemm-programs-new
python3 -B applications/gemm/rvv_benchmark.py execute \
  --programs ../gemm-programs-new --emu "$EMU" --library "$REF" \
  --out ../gemm-results-new --repeat 3 --workers 4 \
  --seconds-limit 7200
python3 -B applications/gemm/rvv_benchmark.py analyze \
  --results ../gemm-results-new --programs ../gemm-programs-new \
  --out ../gemm-analysis-new --repeat 3
```

GEMM 基准脚本固定涵盖 3 种矩阵尺寸、4 种算法实现与 2 种计时口径，共计 24 个裸机程序、各执行 3 次（合计 72 次整核仿真）。执行与分析阶段均会强制核验 DiffTest 状态、128 位或矩阵内存自检结果以及结束 PC 地址。鉴于全量重跑 420 次整核仿真对算力与时间消耗较大，日常只需核对数据时建议优先使用第 12.2 节的离线复查脚本。

13 工程局限与参考资料

13.1 工程局限

1. 真实端到端应用收益待测：目前实验范围集中在微基准点积（取得 3.760303 倍独立吞吐提升）与中小规模热缓存 GEMM 算子，尚未在完整大语言模型（LLM）推理框架、多线程并发、更大矩阵尺寸及冷缓存访存场景下开展实测评估。

2. 全芯片面积占比与物理主频待评估：当前面积与时序数据源自 Nangate 45nm 开源库下的模块级逻辑综合，尚未获取包含昆明湖 V2 全核的同工艺综合总面积，也未开展物理布局布线后的静态时序签核。

3. 标准编码空间迁移待实施：当前硬件仍借用 OPMVV 空余编码点，尚未完成向 RISC-V custom 自定义编码空间的迁移，亦未实现与 Zvqdotq 草案的四字节分组乘累加兼容架构。

4. 进阶微架构改版收益待验证：诸如原位乘累加（读取旧 vd）、单指令多路点积输出、不同流水级数切分以及与 VIMacU 共享乘法器阵列等演进方案，目前仅完成理论评估，尚待后续 RTL 改版与整核实测。

13.2 开源项目与规范

香山处理器与开发环境：https://github.com/OpenXiangShan/XiangShan 和 https://github.com/OpenXiangShan/xs-env

RISC-V 向量扩展规范：https://github.com/riscvarchive/riscv-v-spec ；RISC-V 向量点积扩展草案（Zvqdotq）：https://github.com/riscv/riscv-dot-product 。本设计在指令语义及机器码编码上与上述草案的具体异同分析，详见第二阶段分册第 4 章。